МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп’ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ №8
Варіант 4

на тему «Алгебраїчні типи даних у Python»

Виконав:
студент групи ФеП-22
Корецький Андрій Вікторович

Перевірив:
аспірант
Остріков Данило Андрійович

Львів – 2026

Мета роботи. навчитися моделювати алгебраїчні типи даних через dataclass і Union / TypeAlias, використовувати Literal, Protocol і структурний pattern matching match/case; забезпечити проходження mypy --strict.

Теоретичні відомості

Алгебраїчний тип даних (ADT) типу «сума» описує значення, що є одним із кількох варіантів. У Python варіанти моделюються незмінними класами @dataclass(frozen=True), а сам тип — через Union / TypeAlias (Shape = Circle | Rectangle | Triangle | Group).

Literal звужує допустимі значення (одиниці "mm", "cm", "m"). **Protocol** описує структурний контракт: об'єкт підходить, якщо має потрібні атрибути, без наслідування.

match/case виконує структурний розбір (case Circle(radius=r)), підтримує guard-умови (case Rectangle(width=w, height=h) if w == h). assert_never у гілці case _ дає перевірку вичерпності: mypy повідомить, якщо новий варіант не оброблено.

Індивідуальне завдання (варіант 4)

Тема «Геометричні фігури». ADT: Circle, Rectangle, Triangle. Чистими функціями обчислювати площу та периметр. Literal — для одиниць вимірювання; Protocol — для об'єктів із властивістю area.

Хід роботи

1. Модель ADT

Вузли — frozen dataclass зі slots і валідацією в __post_init__. Group — рекурсивний вузол (фігура з фігур). Unit — Literal; HasArea — Protocol.

```
Unit: TypeAlias = Literal["mm", "cm", "m"]

@dataclass(frozen=True, slots=True)
class Triangle:
    a: float
    b: float
    c: float
    unit: Unit = "cm"

    def __post_init__(self) -> None:
        _require_positive(a=self.a, b=self.b, c=self.c)
        x, y, z = sorted((self.a, self.b, self.c))
        if x + y <= z:
            raise ValueError(
                f"triangle inequality violated: {self.a}, {self.b}, {self.c}"
            )

@dataclass(frozen=True, slots=True)
```

```
class Group:
    """Складена фігура (рекурсивний вузол ADT)."""

    name: str
    shapes: tuple[Shape, ...] = ()
Shape: TypeAlias = Circle | Rectangle | Triangle | Group

class HasArea(Protocol):
    """Структурний контракт: будь-який об'єкт із властивістю ``area``."""

    @property
    def area(self) -> float: ...
```

Лістинг 1 — фрагмент файлу model.py

2. Площа та периметр через match/case

Функції area і perimeter використовують розбір за шаблоном, переводять одиниці та рекурсивно обробляють Group. Гілка case _ з assert_never гарантує вичерпність.

```
def area(shape: Shape, unit: Unit = "cm") -> float:
    """Площа фігури в квадратних ``unit``."""
    match shape:
        case Circle(radius=r, unit=u):
            raw = math.pi * r**2
        case Rectangle(width=w, height=h, unit=u):
            raw = w * h
        case Triangle(a=a, b=b, c=c, unit=u):
            s = (a + b + c) / 2
            raw = math.sqrt(s * (s - a) * (s - b) * (s - c))  # формула Герона
        case Group(shapes=items):
            return sum(area(item, unit) for item in items)
        case _:
            assert_never(shape)
    return raw * length_factor(u, unit) ** 2

def perimeter(shape: Shape, unit: Unit = "cm") -> float:
    """Периметр (для Group — сума периметрів складових)."""
    match shape:
        case Circle(radius=r, unit=u):
            raw = 2 * math.pi * r
        case Rectangle(width=w, height=h, unit=u):
            raw = 2 * (w + h)
        case Triangle(a=a, b=b, c=c, unit=u):
            raw = a + b + c
        case Group(shapes=items):
            return sum(perimeter(item, unit) for item in items)
        case _:
            assert_never(shape)
    return raw * length_factor(u, unit)
```

Лістинг 2 — фрагмент файлу evaluator.py

3. Класифікація з guard-умовами та функції над Protocol

classify розрізняє квадрат, прямокутний, рівносторонній, рівнобедрений і різносторонній трикутники. total_area працює з будь-якими об'єктами, що мають area.

```
def classify(shape: Shape) -> str:
    """Класифікація фігури через match з guard-умовами."""
    match shape:
        case Circle():
            return "circle"
        case Rectangle(width=w, height=h) if math.isclose(w, h):
```

```
            return "square"
        case Rectangle():
            return "rectangle"
        case Triangle(a=a, b=b, c=c) if math.isclose(a, b) and math.isclose(b, c):
            return "equilateral triangle"
        case Triangle(a=a, b=b, c=c) if _is_right(a, b, c):
            return "right triangle"
        case Triangle(a=a, b=b, c=c) if len({a, b, c}) == 2:
            return "isosceles triangle"
        case Triangle():
            return "scalene triangle"
        case Group(shapes=()):
            return "empty group"
        case Group(shapes=items):
            return f"group of {len(items)}"
        case _:
            assert_never(shape)
def total_area(items: Iterable[HasArea]) -> float:
    """Працює з БУДЬ-ЯКИМИ об'єктами, що мають властивість area (structural typing)."""
    return sum(item.area for item in items)
```

Лістинг 3 — фрагмент файлу evaluator.py

4. Тестування

26 тестів: кожен вузол ADT, усі типи трикутників, одиниці, вкладені та порожні групи, scale / convert без мутацій, незмінність і хешованість, помилкові фігури, Protocol зі стороннім класом Plot, pretty-print і базове завдання Expr.

```
def test_nested_group() -> None:
    expected = 12 + 3.0 + 0.64 + math.pi * 0.09  # m²: стіна, дах, вікно, кругле вікно
    assert area(HOUSE, "m") == pytest.approx(expected)
    assert count_shapes(HOUSE) == 4
    assert depth(HOUSE) == 3
    assert classify(HOUSE) == "group of 3"
    assert kind_of(HOUSE) == "group"

def test_protocol_structural_typing() -> None:
    items: list[HasArea] = [measure(Rectangle(2, 3)), Plot("Ivan", 10.0)]
    assert total_area(items) == pytest.approx(16.0)
    assert largest(items) == Plot("Ivan", 10.0)
    assert largest([]) is None
```

Лістинг 4 — фрагмент файлу tests/test_shapes.py

Результати виконання

Результат запуску програми командою `python main.py`:

```
> python main.py
=== Прості фігури (площа та периметр у cm) ===
  Circle(r=2 cm)              circle                S=     12.57  P=  12.57
  Rectangle(4x3 cm)           rectangle             S=     12.00  P=  14.00
  Rectangle(5x5 mm)           square                S=      0.25  P=   2.00
  Triangle(3, 4, 5 cm)        right triangle        S=      6.00  P=  12.00
  Triangle(2, 2, 2 m)         equilateral triangle  S=  17320.51  P= 600.00
  Triangle(5, 5, 8 cm)        isosceles triangle    S=     12.00  P=  18.00

=== Вкладена структура (Group — рекурсивний вузол) ===
house/ (group of 3, S=16.56 m²)
  - Rectangle(4x3 m)  [rectangle]  S=12.00 m²
  - Triangle(4, 2.5, 2.5 m)  [isosceles triangle]  S=3.00 m²
  windows/ (group of 3, S=1.56 m²)
    - Rectangle(80x80 cm)  [square]  S=0.64 m²
    - Rectangle(80x80 cm)  [square]  S=0.64 m²
    - Circle(r=30 cm)  [circle]  S=0.28 m²
  render: Group[house: Rectangle(4x3 m), Triangle(4, 2.5, 2.5 m), Group[windows:
```

```
      Rectangle(80x80 cm), Rectangle(80x80 cm), Circle(r=30 cm)]]
  фігур: 5 | глибина: 3
  площа, m²: 16.5627
  measure  : Measurement(kind='group', area=16.5627, perimeter=31.285, unit='m')

=== Pure-перетворення ===
  scale(Circle(2), 3) -> Circle(r=6 cm) | оригінал: Circle(r=2 cm)
  convert(Rectangle(4x3 m), 'cm') -> Rectangle(400x300 cm)

=== Protocol HasArea ===
  total_area(measurements) = 17363.32
  largest(measurements)    = Measurement(kind='triangle', area=17320.5081, perimeter=600.0,
      unit='cm')

=== Помилкові дані ===
  ValueError: triangle inequality violated: 1, 2, 10
  ValueError: radius must be positive, got -1
=== Базове завдання: Expr = Num | Add | Mul ===
  ((2 + 3) * 4) = 20
```

Рисунок 1 — результат роботи програми (Lab_08)

Результат автоматизованого тестування `pytest -v` та статичних перевірок:

```
> black --check .
All done!
> ruff check .
All checks passed!
> mypy .
Success: no issues found
> pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.3, pytest-9.2.0, pluggy-1.7.0 -- C:\labs\Функціональне
    програмування\.venv\Scripts\python.exe
rootdir: C:\labs\Функціональне програмування\functional-programming\Lab_08
configfile: pyproject.toml
testpaths: tests
plugins: platformdirs-4.14.5
collecting ... collected 26 items
tests/test_expr.py::test_eval_basic PASSED                               [  3%]
tests/test_expr.py::test_eval_nested PASSED                              [  7%]
tests/test_expr.py::test_render PASSED                                   [ 11%]
tests/test_expr.py::test_overflow PASSED                                 [ 15%]
tests/test_shapes.py::test_circle PASSED                                 [ 19%]
tests/test_shapes.py::test_rectangle_and_square PASSED                   [ 23%]
tests/test_shapes.py::test_triangles[tri0-6.0-right triangle] PASSED     [ 26%]
tests/test_shapes.py::test_triangles[tri1-1.7320508075688772-equilateral triangle] PASSED [
    30%]
tests/test_shapes.py::test_triangles[tri2-12.0-isosceles triangle] PASSED [ 34%]
tests/test_shapes.py::test_triangles[tri3-9.921567416492215-scalene triangle] PASSED [ 38%]
tests/test_shapes.py::test_units_conversion PASSED                       [ 42%]
tests/test_shapes.py::test_convert_returns_new_equivalent_shape PASSED   [ 46%]
tests/test_shapes.py::test_nested_group PASSED                           [ 50%]
tests/test_shapes.py::test_empty_group_edge_case PASSED                  [ 53%]
tests/test_shapes.py::test_scale_is_pure_and_recursive PASSED            [ 57%]
tests/test_shapes.py::test_nodes_are_frozen_and_hashable PASSED          [ 61%]
tests/test_shapes.py::test_invalid_shapes[<lambda>0] PASSED              [ 65%]
tests/test_shapes.py::test_invalid_shapes[<lambda>1] PASSED              [ 69%]
tests/test_shapes.py::test_invalid_shapes[<lambda>2] PASSED              [ 73%]
tests/test_shapes.py::test_invalid_shapes[<lambda>3] PASSED              [ 76%]
tests/test_shapes.py::test_invalid_shapes[<lambda>4] PASSED              [ 80%]
tests/test_shapes.py::test_invalid_shapes[<lambda>5] PASSED              [ 84%]
tests/test_shapes.py::test_protocol_structural_typing PASSED             [ 88%]
tests/test_shapes.py::test_measure PASSED                                [ 92%]
tests/test_shapes.py::test_render PASSED                                 [ 96%]
tests/test_shapes.py::test_render_tree_is_nested PASSED                  [100%]
============================== 26 passed in 0.08s ==============================
```

Рисунок 2 — результати тестів і перевірок якості коду (Lab_08)

Висновок

Створено ADT геометричних фігур на основі незмінних dataclass-вузлів і TypeAlias-об'єднання, включно з рекурсивним вузлом Group. Обчислення площі, периметра, масштабування, переведення одиниць, класифікація та текстове представлення реалізовані чистими функціями з match/case без ланцюгів isinstance. Literal обмежує одиниці вимірювання, Protocol HasArea забезпечує структурну типізацію. Проєкт проходить mypy --strict, black, ruff і всі тести.